

HI-TECH ACADEMY · ORGANISME DE FORMATION

IA for Tech

Concevez, construisez et déployez
des applications IA de niveau production

Support de cours — V1.1 du 08/10/2026 — 100 h — 100 % à distance

Formateur : Mahdi CHEKINI — contact@hi-techacademy.fr — 07 51 47 41 35

Organisation de la formation

- 100 heures en séances à distance — planning défini à l'inscription, sur votre convocation.
- Environnements pré-configurés : vous codez dès la première heure (clés API et infra de TP fournies).
- **Projet final construit brique par brique : une application agentique complète (RAG + MCP + evals), déployée et évaluée.**
- Émargement par demi-journée + rapport de connexion Google Meet.

OBJECTIFS

À l'issue de la formation, vous saurez...

- 1. Maîtriser les fondamentaux LLM côté ingénierie (contexte, inférence, open-source, context engineering).
- 2. Intégrer les LLM proprement en production (APIs, retries, coûts, tool use, sorties structurées).
- 3. Construire du RAG du prototype au RAG agentique — et savoir quand fine-tuner.
- 4. Concevoir agents et systèmes multi-agents : LangGraph, Agent SDKs, MCP, sandboxing.
- 5. Livrer fiable, sûr et conforme — et travailler en équipe augmentée (Claude Code, spec-driven, CI/CD).

PROGRAMME

Le parcours (100 h, 6 modules + projet final)

- M1 Fondamentaux (14 h) → M2 APIs & SDKs (16 h) → M3 RAG (18 h)
- M4 Agents & orchestration (20 h) → M5 Qualité, sécurité, production (16 h) → M6 IA en équipe (11 h)
- Clôture (3 h) : QCM /10 — le projet final compte pour la partie pratique /10 (seuil 12/20).
- **Le projet final avance à chaque module : RAG (M3) → agent + MCP (M4) → evals + guardrails (M5) → déploiement (M6).**

MODULE 1 · 14 HEURES

Fondamentaux

Les bases solides, au juste niveau de profondeur.

Architecture des LLM — le juste niveau

- Transformer : attention sur toute la séquence — pourquoi le contexte coûte cher ($O(n^2)$).
- Pré-entraînement (prédire le token suivant) puis post-training (instructions, RLHF).
- Ce qu'il faut en retenir pour coder : un LLM est une fonction texte $\rightarrow$ distribution de tokens.
- Pas besoin d'entraîner un modèle pour construire des produits IA — il faut savoir les piloter.

Tokens, embeddings, contexte : dans votre code

- Tokens : unité de facturation ET de limite — comptez-les (tiktoken et équivalents).
- Embeddings : vecteurs qui capturent le sens — la brique de la recherche sémantique (module 3).
- **Fenêtre de contexte : tout ce que le modèle voit. Trop remplir dégrade la qualité ET le coût.**
- TP : mesurer tokens et coûts réels de vos prompts types.

Paramètres d'inférence — ce que chacun change

- temperature : 0 pour l'extraction/classification, plus haut pour la créativité.
- top_p : alternative au sampling par température — n'ajustez qu'un des deux.
- max_tokens : plafond de sortie — dimensionnez pour éviter les coupures.
- streaming : UX indispensable dès qu'un humain attend la réponse.

TP : même prompt, grille de paramètres — observez les variations de sortie.

Panorama des modèles : capacités, coûts, latence

- Frontier (Claude, GPT, Gemini) : raisonnement, agents, multimodal — plus chers, plus lents.
- Petits modèles (Haiku, mini, open-source) : classification, extraction, routage — centimes et millisecondes.
- **Architecture type : petit modèle pour 80 % du volume, grand modèle pour les cas durs (routing, module 5).**
- Grille de choix : qualité requise × volume × latence × budget.

Open-source et déploiement local

- Mistral, Qwen, Llama : le niveau monte vite — souvent suffisant pour des tâches ciblées.
- Ollama : un LLM sur votre machine en une commande — idéal dev, démos, données sensibles.
- Cas d'usage souveraineté : données qui ne doivent pas sortir, environnements isolés.
- TP : servir un modèle local avec Ollama et l'appeler depuis votre code.

Prompting pour développeurs

- System prompt : rôle, règles, format — le contrat de votre feature.
- Sorties structurées : réclamez du JSON/XML avec schéma explicite (validation au module 2).
- Balises XML pour délimiter les entrées : robustesse contre le contenu imprévisible.
- Few-shot : 2-3 exemples valent mieux qu'un paragraphe d'explications.

Context engineering — la vraie compétence

- **La qualité d'une app LLM se joue dans CE QUI ENTRE dans le contexte : sélection, ordre, format.**
- Trop de contexte : dilution, coût, latence. Trop peu : hallucinations.
- Techniques : résumés progressifs, sélection dynamique, structuration par sections.
- Le RAG (module 3) et les agents (module 4) sont du context engineering appliqué.

TP fil rouge — l'assistant de bord

- Construisez un petit CLI qui interroge un LLM avec system prompt et sortie structurée.
- Variantes : modèle frontier vs petit modèle vs local — comparez qualité, coût, latence.
- Ce squelette servira de base aux TP des modules suivants.

Lire une facture d'API sans surprise

- Prix par million de tokens, entrée vs sortie : la sortie coûte souvent 3 à 5 fois plus.
- Le contexte se re-paie à chaque tour de conversation — d'où l'intérêt des résumés.
- Batch API : -50 % pour les traitements différés.
- Exercice : estimez le coût mensuel de 3 features types (chat, extraction, résumé).

Reasoning models : quand les utiliser

- Les modèles « à raisonnement » réfléchissent avant de répondre : meilleurs sur maths, code complexe, planification.
- Coût et latence supérieurs : réservez-les aux problèmes qui le justifient.
- Le budget de réflexion se pilote (effort/thinking budget selon les APIs).
- Règle : modèle standard d'abord ; escalade vers le raisonnement si l'eval le justifie.

Structurer ses prompts : le gabarit

- Ordre robuste : rôle → contexte/documents (balisés) → tâche → contraintes → format de sortie.
- Instructions positives (« fais X ») plutôt que négatives (« ne fais pas Y »).
- Le contenu variable TOUJOURS délimité : balises XML ou fences — jamais concaténé nu.
- Versionnez vos prompts comme du code : ils en sont.

TP — le banc d'essai de modèles

- Constituez 10 cas de test représentatifs de VOTRE domaine.
- Exécutez-les sur 3 modèles (frontier, petit, local) — script fourni.
- Tableau comparatif : qualité perçue, latence, coût pour 1 000 exécutions.
- Première brique de votre culture evals (module 5).

Récap module 1 — quiz

- Pourquoi la fenêtre de contexte est-elle une contrainte de conception ?
- Quand un petit modèle suffit-il ?
- Que mettre dans un system prompt ?
- Context engineering : donnez une définition en une phrase.

MODULE 2 · 16 HEURES

APIs et SDKs

Intégrer les LLM proprement dans votre code.

Les APIs LLM : patterns communs

- Le pattern messages : system / user / assistant — commun à Anthropic, OpenAI, Gemini.
- Différences qui comptent : gestion du system prompt, formats de tool use, streaming.
- Clés API : variables d'environnement, jamais dans le code ni dans Git.
- TP : le même appel sur deux providers — repérez les différences.

SDKs en production (1/2) — les erreurs arrivent

- 429 (rate limit) : backoff exponentiel + jitter, gestion de quota par utilisateur.
- 5xx et timeouts : retries bornés, puis dégradation propre (message, file d'attente).
- **Règle : tout appel LLM peut échouer — votre UX doit y survivre.**
- TP : wrapper robuste avec retries, timeout et journalisation.

SDKs en production (2/2) — maîtriser les coûts

- Loggez tokens entrée/sortie par appel : c'est votre compteur électrique.
- Prompt caching : les préfixes répétés (system, docs) coûtent une fraction du prix.
- Plafonds : budget par utilisateur/fonctionnalité, alertes de dépassement.
- Estimation d'un coût de feature AVANT de la lancer : $\text{tokens} \times \text{volume} \times \text{prix}$.

Tool use / function calling (1/2)

- Vous déclarez des fonctions (nom, description, schéma) ; le modèle demande leur exécution.
- **C'est VOTRE code qui exécute — le modèle ne fait que demander. Vous gardez le contrôle.**
- La boucle : appel → tool_use → exécution → tool_result → réponse finale.
- Qualité des descriptions d'outils = qualité des décisions du modèle.

Tool use / function calling (2/2) — TP

- Implémentez 2 outils : recherche dans une base locale + appel d'API météo/exchange.
- Gérez la boucle complète, y compris les erreurs d'outil renvoyées au modèle.
- Observez : quand le modèle choisit-il d'appeler, de combiner, de s'abstenir ?

Multimodal en pratique

- Vision : screenshots, schémas, photos de documents — extraction et analyse.
- PDF : envoyés nativement aux APIs qui les supportent, ou convertis.
- Audio : transcription puis traitement — le pattern voix → texte → LLM.
- TP : extraire les données structurées d'une facture scannée.

Sorties structurées et validation

- JSON mode / structured outputs : le modèle contraint au schéma.
- **Validez TOUJOURS côté code : Pydantic (Python) / Zod (TypeScript) — le schéma est un contrat.**
- En cas d'échec de validation : re-demander avec l'erreur en contexte (self-healing).
- TP : pipeline extraction → validation → retry automatique.

TP de synthèse — le service d'extraction

- Construisez un endpoint qui reçoit un document et renvoie du JSON validé.
- Exigences : retries, gestion du rate limit, log des coûts, schéma Pydantic/Zod.
- Revue de code croisée entre participants.

Production évaluée : service robuste démontré sur les documents de test.

Streaming côté serveur et côté client

- SSE (server-sent events) : le pattern standard pour relayer le flux au navigateur.
- Affichage progressif + interruption utilisateur : les deux exigences UX.
- Attention aux transformations en flux (JSON partiel) : parsez en fin de stream ou par blocs.
- TP : endpoint streaming complet, du provider au navigateur.

Concurrence et files d'attente

- Parallélisez les appels indépendants (asyncio / Promise.all) — la latence est votre ennemie.
- File d'attente pour lisser les pics et respecter vos quotas providers.
- Idempotence : un retry ne doit pas dupliquer un effet de bord.
- TP : traiter 100 documents en parallèle borné, sans dépasser le rate limit.

Choisir le bon pattern d'intégration

- Requête simple : pour les transformations one-shot (résumé, extraction).
- Conversation avec état : sessions persistées côté serveur, résumé du contexte long.
- Tool use en boucle : dès que le modèle doit consulter vos systèmes.
- Traitement par lots : volumes importants sans contrainte temps réel (batch API).

Cas réel — anatomie d'une feature LLM bien intégrée

- Lecture guidée d'une implémentation de référence (dépôt fourni).
- Ce qu'on y trouve : wrapper robuste, schémas validés, logs de coûts, tests.
- Ce qu'on n'y trouve pas : prompts en dur dans le code, clés en clair, appels sans timeout.

Récap module 2 — quiz

- Que faire face à un 429 ?
- Qui exécute le code dans le tool use ?
- Pourquoi valider les sorties même en JSON mode ?
- Deux leviers pour réduire la facture API ?

MODULE 3 · 18 HEURES

RAG

Du premier pipeline au RAG agentique.

RAG de A à Z — l'architecture

- Ingestion : découper les documents (chunking), calculer les embeddings, indexer.
- Requête : embedder la question, récupérer les k chunks les plus proches, les donner au modèle.
- **Promesse : des réponses ancrées dans VOS documents, avec sources — moins d'hallucinations.**
- TP : premier pipeline complet sur corpus fourni, en moins de 100 lignes.

Chunking : l'étape sous-estimée

- Taille des chunks : trop petits = contexte perdu ; trop gros = bruit et dilution.
- Découpez selon la structure (titres, sections), pas au caractère près.
- Métadonnées (source, section, date) : indispensables pour filtrer et citer.
- TP : comparez 3 stratégies de chunking sur le même corpus — mesurez l'impact.

Bases vectorielles et retrieval

- Options : pgvector (déjà dans votre Postgres), bases dédiées, index en mémoire pour prototyper.
- Similarité cosinus, k-NN : ce qu'il faut comprendre, sans plus.
- Le retrieval est le maillon faible n° 1 : si les bons chunks ne remontent pas, rien ne sauve la réponse.
- Mesurez le retrieval séparément de la génération (recall@k).

RAG avancé (1/2) — hybrid search et reranking

- Hybrid search : vecteurs + mots-clés (BM25) — robuste aux termes exacts (références, noms).
- Reranking : un second modèle réordonne les résultats — gros gain pour peu d'effort.
- Query rewriting : reformuler/décomposer la question avant le retrieval.
- TP : ajoutez hybrid + reranking à votre pipeline, mesurez le gain.

RAG avancé (2/2) — GraphRAG et au-delà

- GraphRAG : extraire entités et relations pour répondre aux questions « globales » (synthèses).
- Coût de construction élevé : réservé aux corpus où les relations comptent.
- Multi-index : router la question vers le bon corpus avant le retrieval.
- Le bon RAG est le plus simple qui atteint vos métriques.

Agentic RAG

- **L'agent décide : chercher ou non, dans quelle source, reformuler, itérer si le résultat est faible.**
- Pattern : retrieval as a tool — la recherche devient un outil parmi d'autres.
- Gains sur les questions complexes ; coût en latence et en tokens.
- TP : transformez votre pipeline en agentic RAG avec le tool use du module 2.

RAG vs fine-tuning vs long contexte

- Long contexte : corpus petit et stable → tout mettre dans le prompt (avec caching).
- RAG : connaissances volumineuses, changeantes, à sourcer — le défaut raisonnable.
- Fine-tuning : style, format, tâche spécialisée — PAS pour injecter des connaissances.
- Framework de décision fourni : volume × fraîcheur × traçabilité × budget.

Fine-tuning en pratique

- LoRA : adapter un modèle sans réentraîner tous les poids — accessible en coût.
- Le vrai travail : le dataset (qualité, diversité, format) — pas l'entraînement.
- Quand ça vaut le coup : tâche répétitive à très fort volume, style imposé, petit modèle spécialisé.
- Démo : fine-tuning LoRA d'un petit modèle sur une tâche de classification.

Projet final — jalon RAG

- **Posez le pipeline RAG de VOTRE application : corpus choisi, ingestion, retrieval mesuré.**
- Exigences : chunking raisonné, métadonnées, réponses sourcées, recall mesuré sur 10 questions de test.
- Revue individuelle avec le formateur.

Jalon projet : pipeline RAG posé et mesuré.

Évaluer son RAG : les métriques qui comptent

- Retrieval : recall@k, precision@k — les bons chunks remontent-ils ?
- Génération : fidélité aux sources (pas d'invention), complétude, citation correcte.
- Bout en bout : taux de bonnes réponses sur votre jeu de questions de référence.
- Mesurez avant/après chaque changement : chunking, embeddings, reranking.

Ingestion continue et fraîcheur

- Pipeline d'ingestion automatisé : nouveaux documents indexés sans intervention.
- Mises à jour : ré-indexer le document modifié, invalider l'ancien — pas tout le corpus.
- Horodatage dans les métadonnées : privilégier les sources récentes au retrieval.
- TP : ingestion incrémentale sur votre corpus projet.

Les pièges classiques du RAG

- Chunks sans contexte : ajoutez titre du document et de la section à chaque chunk.
- Question hors corpus : le système doit dire « je ne sais pas », pas broder.
- Tableaux et données structurées : le découpage naïf les détruit — traitement dédié.
- Multilinguisme : embeddings et requêtes dans des langues différentes.

Cas réel — un RAG de support client disséqué

- Architecture complète étudiée : ingestion FAQ + tickets, hybrid search, reranking, citations.
- Ses métriques : recall 0,85 → 0,93 après reranking ; fidélité jugée par LLM 4,6/5.
- Ses choix : pgvector plutôt qu'une base dédiée, pourquoi c'était le bon appel ici.

Récap module 3 — quiz

- Le maillon faible n° 1 d'un RAG ?
- Hybrid search : pourquoi combiner vecteurs et mots-clés ?
- RAG ou fine-tuning pour des connaissances qui changent chaque semaine ?
- Qu'est-ce que l'agentic RAG change ?

MODULE 4 · 20 HEURES

Agents et orchestration

Concevoir des systèmes qui agissent.

Architectures agentiques — les patterns

- ReAct : raisonner → agir → observer, en boucle jusqu'à l'objectif.
- Planning : décomposer d'abord, exécuter ensuite — meilleur sur les tâches longues.
- Mémoire : court terme (contexte), long terme (store externe), épisodique (historique).
- **Un agent = LLM + outils + boucle + critères d'arrêt. Le reste est de l'ingénierie autour.**

Boucles et critères d'arrêt

- Budget d'itérations et de tokens : un agent sans limite est une facture sans limite.
- Conditions de sortie explicites : objectif atteint, échec assumé, escalade humaine.
- Journalisez chaque étape : indispensable pour déboguer un comportement émergent.
- TP : boucle ReAct maison avec budget et journal, sur vos outils du module 2.

LangGraph — workflows stateful

- Le graphe : nœuds (étapes) + arêtes (transitions), état typé partagé.
- Contrôle fin : branchements, retours en arrière, interruptions pour validation humaine.
- Persistance : reprendre un workflow interrompu — indispensable en production.
- TP : workflow LangGraph à 4 nœuds avec human-in-the-loop.

Le paysage 2026 des frameworks

- Agent SDKs des fournisseurs (Claude, OpenAI) : intégration native, montent en puissance.
- PydanticAI : typage fort, léger — élégant pour les services structurés.
- CrewAI : multi-agents « rôles » rapide à prototyper.
- **Critère de choix : commencez par le SDK du fournisseur ou un framework léger ; n'ajoutez de l'abstraction que si elle paie.**

MCP (1/3) — le protocole devenu standard

- Un serveur MCP expose outils, ressources et prompts ; tout client compatible les consomme.
- Découplage : vos intégrations écrites une fois, utilisables par Claude, IDEs, agents.
- Transport stdio ou HTTP ; découverte dynamique des capacités.
- TP : brancher des serveurs MCP existants (filesystem, base de données) sur votre client.

MCP (2/3) — construire votre serveur

- SDK officiel : déclarer des outils typés avec schémas, c'est l'essentiel du travail.
- Bonnes pratiques : descriptions précises, erreurs explicites, permissions minimales.
- TP : votre serveur MCP qui expose votre domaine métier (3 outils).
- Ce serveur rejoint votre projet final.

MCP (3/3) — publier et sécuriser

- Publication : packaging, versionnage, documentation des outils.
- Sécurité : authentification, scopes, journalisation des appels.
- **Un serveur MCP est une surface d'attaque : traitez-le comme une API publique.**

Systèmes multi-agents

- Pattern orchestrateur : un agent décompose et délègue à des sous-agents spécialisés.
- Communication : résultats structurés, pas de prose entre agents.
- Quand ça vaut le coup : sous-tâches réellement parallèles ou expertises distinctes.
- Piège : la complexité explose — deux agents qui marchent battent cinq qui divergent.

Sandboxing et exécution de code

- **Un agent qui exécute du code généré = exécution de code arbitraire : isolez TOUJOURS.**
- Options : conteneurs jetables, VMs légères, environnements restreints (réseau, filesystem).
- Moindre privilège : l'agent n'accède qu'à ce dont il a besoin, temporairement.
- TP : exécution sandboxée d'un code généré, avec limites CPU/réseau.

Projet final — jalon agent + MCP

- **Intégrez à votre application : un agent (LangGraph ou SDK) + votre serveur MCP.**
- Exigences : boucle bornée, validation humaine sur les actions sensibles, journal des étapes.
- Revue individuelle avec le formateur.

Jalon projet : agent et serveur MCP intégrés.

Human-in-the-loop : les patterns

- Interruption avant action sensible : l'agent s'arrête, l'humain approuve, le workflow reprend.
- Files de validation : actions en attente groupées, revue par lots.
- Seuils de confiance : en-dessous, escalade automatique.
- LangGraph : interrupts et checkpoints rendent ces patterns naturels.

La mémoire des agents en pratique

- Court terme : l'état du graphe / la conversation en cours.
- Long terme : faits persistés en base (préférences, historique) réinjectés au besoin.
- Résumés périodiques pour contenir la croissance du contexte.
- TP : donner une mémoire long terme à votre agent (store + rappel sélectif).

Déboguer un agent : méthode

- Rejouez depuis le journal : chaque étape, chaque décision d'outil, chaque prompt réel.
- Isolez : testez l'outil seul, le prompt seul, puis la boucle.
- Les échecs récurrents deviennent des cas d'eval (module 5).
- Un agent non traçable est un agent non débogable.

Coûts des agents : garder le contrôle

- Une boucle agentique multiplie les appels : suivez le coût PAR TÂCHE, pas par appel.
- Optimisations : petit modèle pour les décisions simples, caching du contexte stable, arrêts précoces.
- Budget par exécution avec coupure propre + reprise possible.

Cas réel — un agent de production disséqué

- Étude d'un agent de traitement de commandes : graphe, outils, validations humaines.
- Ses garde-fous : budget 15 itérations, actions d'écriture toutes validées, journal complet.
- Ses métriques : 92 % de tâches abouties sans intervention, coût moyen 0,08 € / tâche.

Récap module 4 — quiz

- Les 4 composants d'un agent ?
- Pourquoi des critères d'arrêt explicites ?
- Ce qu'apporte MCP par rapport à des intégrations ad hoc ?
- Pourquoi sandboxer l'exécution de code généré ?

MODULE 5 · 16 HEURES

Qualité, sécurité, production

Livrer des systèmes fiables, sûrs et conformes.

Evals (1/3) — la compétence la plus demandée

- **Sans evals, vous ne savez pas si un changement de prompt ou de modèle améliore ou casse.**
- Dataset de test : des cas réels + cas limites + cas adverses, avec réponses attendues.
- Métriques : exactitude, fidélité aux sources (RAG), format, refus appropriés.
- Commencez petit : 20 bons cas de test valent mieux que zéro.

Evals (2/3) — LLM-as-judge

- Un modèle note les sorties selon une grille explicite (critères, échelle, exemples).
- Calibrez le juge : vérifiez son accord avec vos jugements humains sur un échantillon.
- Coût maîtrisé : jugez par échantillonnage en continu, en totalité avant release.
- TP : grille de juge + éval automatisée de votre pipeline RAG.

Evals (3/3) — dans le cycle de dev

- Evals en CI : chaque changement de prompt/modèle passe la suite avant merge.
- Régression tracking : les scores dans le temps, par version.
- Boucle : échecs d'eval → nouveaux cas de test → amélioration ciblée.

Observabilité des apps LLM

- Tracez chaque requête : prompt, contexte, réponse, tokens, latence, coût, verdict d'eval.
- **Supervision et monitoring des agents : traces d'exécution (étapes, outils appelés), métriques, coûts, alertes sur les échecs.**
- Outils : plateformes de tracing LLM dédiées ou votre stack d'observabilité étendue.
- Les traces sont votre matière première : débogage, datasets d'eval, détection de dérive.
- TP : instrumenter votre application avec un tracing complet.

Sécurité LLM (1/2) — connaître les attaques

- **Prompt injection : du contenu (page web, document, e-mail) détourne les instructions de votre agent.**
- Jailbreaks : contourner les règles du modèle ; exfiltration de system prompt et de données.
- OWASP LLM Top 10 : la carte des risques — revue guidée.
- Démonstration : injection réussie sur un agent naïf... puis contrée.

Sécurité LLM (2/2) — guardrails

- Entrées : filtrage, détection d'injection, quotas par utilisateur.
- Sorties : validation de schéma, filtres de contenu, vérification des actions demandées.
- Architecture : moindre privilège des outils, données sensibles hors contexte, validation humaine sur l'irréversible.
- TP : ajouter des guardrails mesurables à votre application (bloquer les cas adverses du dataset).

RGPD et AI Act dans vos architectures

- RGPD : minimisation des données dans les prompts, DPA avec les fournisseurs, durée de rétention des traces.
- AI Act : transparence (l'utilisateur sait qu'il parle à une IA), documentation des systèmes, gestion par niveau de risque.
- Anonymisez avant l'appel API quand c'est possible ; attention aux logs qui capturent tout.
- Checklist conformité fournie — à dérouler sur le projet final.

Architecture de production

- Caching : préfixes (prompt caching) et réponses (cache sémantique) — jusqu'à -90 % de coûts.
- Fallbacks : modèle de secours, dégradation propre, files de retry.
- Routing multi-modèles : petit modèle par défaut, escalade vers le grand sur critères.
- TP : ajouter caching + fallback + routing à votre application.

Projet final — jalon qualité et sécurité

- **Votre application passe au niveau production : evals, guardrails, tracing en place.**
- Exigences : suite d'evals exécutable, 3 guardrails démontrés, traces consultables.
- Revue individuelle avec le formateur.

Jalon projet : evals + guardrails + observabilité.

Construire son dataset d'evals

- Sources : vos traces réelles (anonymisées), cas imaginés, cas adverses, tickets d'incident.
- Format : entrée, sortie attendue (ou critères), tags par catégorie.
- Vivez avec : chaque bug de prod devient un cas de test.
- TP : 20 cas pour votre application, répartis par objectif.

Détection de dérive en production

- Les modèles évoluent (mises à jour provider) : vos scores peuvent bouger sans que vous changiez rien.
- Éval continue par échantillonnage + alerte si chute de score.
- Suivez aussi les signaux utilisateurs : taux de retry, feedback, abandons.

Atelier rouge/bleu — attaque et défense

- Équipe rouge : injecter, jailbreaker, exfiltrer sur l'app d'un autre binôme.
- Équipe bleue : détecter, bloquer, journaliser.
- Débrief : chaque attaque réussie devient un guardrail + un cas d'eval.

L'exercice le plus formateur du module — en environnement isolé fourni.

Gérer les données sensibles

- PII : détection et masquage AVANT l'appel API quand c'est possible.
- Logs et traces : ne capturez pas ce que vous n'avez pas le droit de conserver.
- Rétention : purge automatique alignée sur votre politique RGPD.
- Contrats fournisseurs : zero-retention / non-entraînement à vérifier.

Checklist mise en production

- Evals au vert sur la suite complète ; guardrails testés contre le dataset adverse.
- Tracing, alertes de coût et de dérive actifs.
- Fallbacks testés (panne provider simulée) ; conformité RGPD/AI Act déroulée.
- Runbook : qui fait quoi quand ça casse.

Récap module 5 — quiz

- Pourquoi les evals sont-elles non négociables ?
- Comment calibrer un LLM-as-judge ?
- Deux contre-mesures à la prompt injection ?
- Trois leviers d'architecture pour réduire coûts et pannes ?

MODULE 6 • 11 HEURES

Travailler avec l'IA en équipe

Le développeur orchestrateur.

Claude Code — maîtrise au quotidien

- Au-delà de l'autocomplétion : explorer une base de code, refactorer, déboguer, générer des tests.
- Le contexte fait tout : CLAUDE.md, conventions, accès aux bons fichiers.
- Modes de travail : exploration → plan → exécution — gardez la main sur les étapes.
- TP : résoudre un ticket réel de bout en bout avec l'assistant.

Spec-driven development

- **La spec d'abord : critères d'acceptation clairs, PUIS l'IA implémente — pas l'inverse.**
- Une bonne spec : contexte, comportement attendu, cas limites, contraintes, tests.
- L'IA aide aussi à écrire la spec : challenge, cas oubliés, ambiguïtés.
- TP : spec → implémentation par l'IA → revue contre la spec.

Workflows d'équipe

- CLAUDE.md : les conventions du projet, lues par l'assistant de chaque développeur.
- Revues de code IA : premier passage automatique, l'humain arbitre.
- Contexte partagé : documentation à jour = assistants efficaces pour toute l'équipe.
- Convention d'équipe : ce que l'IA peut faire seule, ce qui exige une revue humaine.

Agents parallèles

- Déléguer plusieurs tâches indépendantes simultanément (tests, doc, refactor).
- Isolation : chaque agent sur sa branche/worktree — pas de conflits.
- Votre rôle : découper, briefer, superviser, intégrer.
- TP : deux tâches menées en parallèle par deux agents, revue et merge.

L'IA dans la CI/CD

- Tests générés sur le diff ; revues automatiques commentées sur la PR.
- Documentation maintenue : l'agent met à jour docs et changelogs.
- Garde-fous : l'IA propose, la CI vérifie (tests, lint, evals), l'humain merge.

Le nouveau rôle du développeur

- La valeur se déplace : de l'écriture du code vers la spécification, l'architecture et la revue.
- Compétences qui montent : découpage des problèmes, evals, sécurité, orchestration.
- **Vous restez responsable de ce que vous livrez — l'IA n'endosse rien.**

Projet final — déploiement

- Déployez votre application agentique complète (cloud ou environnement fourni).
- Checklist : RAG + agent + MCP + evals + guardrails + tracing, accessibles et démontrables.
- Préparez la démonstration : 10 minutes, en conditions réelles.

Démonstrations individuelles

- **Évaluation /10 sur la grille officielle : RAG (2), tool use / MCP (2), evals (2), sécurité (2), déploiement + observabilité (2).**
- Chaque démo : architecture, parcours utilisateur, un cas difficile gérés en direct.
- Retour individuel du formateur : points forts, axe de progression prioritaire.

Partie pratique de l'évaluation finale — notée pendant les démonstrations.

Bien briefier un assistant de code

- Contexte minimal utile : fichiers concernés, contrainte, définition du fini.
- Demandez le plan avant l'implémentation sur les tâches non triviales.
- Exigez les tests avec le code — et faites-les tourner.
- Relisez comme une PR humaine : vous signez ce qui part en prod.

Écrire un bon CLAUDE.md

- Commandes du projet (build, test, lint), architecture en 10 lignes, conventions de code.
- Les pièges connus du repo : ce qui casse souvent, ce qu'il ne faut pas toucher.
- Court et à jour vaut mieux qu'exhaustif et périmé.
- TP : rédigez le CLAUDE.md de votre projet final.

Cas réel — une équipe augmentée au quotidien

- Rituel : specs le matin, agents en parallèle la journée, revues humaines groupées.
- Leurs métriques : lead time divisé par 2, revues plus exigeantes, docs enfin à jour.
- Leurs règles : jamais de merge sans humain, evals en CI, budget IA suivi par squad.

TP libre — finitions du projet accompagnées

- Temps de travail individuel sur le projet final, formateur en soutien.
- Priorité aux critères de la grille encore fragiles.
- Répétition express de la démo pour ceux qui le souhaitent.

Récap module 6 — quiz

- Spec-driven : qu'est-ce qui vient en premier ?
- À quoi sert CLAUDE.md dans une équipe ?
- Que vérifie la CI quand l'IA contribue ?
- Où se déplace la valeur du développeur ?

CLÔTURE · 3 HEURES

Évaluation finale et feuille de route

Valider les acquis, continuer en production.

Évaluation finale — modalités

- Partie A — QCM : 10 questions, /10, une seule tentative (lien envoyé par e-mail).
- Partie B — projet final : déjà noté /10 lors des démonstrations du module 6.
- **Objectifs atteints à partir de 12/20 — résultats reportés sur votre attestation de fin de formation.**
- Correction commentée collective après le QCM.

CLÔTURE

Merci — et bonnes mises en production

- Feuille de route individuelle : prochaines briques de votre application, veille à cadence tenable.
- Questionnaire de satisfaction à chaud avant de partir ; à froid dans quelques semaines.
- Attestation de fin de formation envoyée après la session.
- **Contact : Mahdi CHEKINI — contact@hi-techacademy.fr — 07 51 47 41 35**

Glossaire

Terme	Définition
Token / contexte	Unité de texte ; la fenêtre de contexte borne ce que le modèle traite.
Embedding	Vecteur représentant le sens d'un texte — base de la recherche sémantique.
Tool use	Le modèle demande l'exécution de fonctions définies par le développeur.
RAG / agentic RAG	Réponses ancrées sur documents récupérés ; l'agent pilote la recherche.
MCP	Protocole standard pour connecter modèles, outils et sources de données.
Evals / LLM-as-judge	Suites de tests des sorties LLM ; un modèle peut servir de juge calibré.
Prompt injection	Contenu malveillant qui détourne les instructions — contrôlé par guardrails.
Guardrails	Contrôles d'entrées/sorties et de permissions autour d'un système LLM.

Ressources pour continuer

- Documentations officielles : Anthropic (API, MCP, Claude Code), OpenAI, LangGraph, Ollama.
- OWASP LLM Top 10 : la référence sécurité, mise à jour régulièrement.
- CNIL et Commission européenne : RGPD appliqué à l'IA, textes AI Act.
- Vos dépôts de TP et le projet final : la meilleure base pour vos prochains produits.
- Hi-Tech Academy — <https://hi-tech-academy.fr/> : support et contact formateur.